POLARIS
SCHOOL OF TECHNOLOGY

GENERATIVE AI - ARCHITECTURE STUDY NOTES

Autoregressive & Encoder-Decoder Models

Design intuition, training mechanics, cross-attention, output layers, and worked examples for choosing and building sequence models.



Core distinction. Autoregressive describes how outputs are factorized and generated one token at a time. Encoder-decoder describes an architecture in which one network first represents the source and a second network generates the target while attending to that source representation.

Question	Short answer
What does an encoder output?	A contextual vector for every source position - often called encoder memory.
What does the decoder output?	A hidden vector for every target position; the final projection turns it into vocabulary logits.
Where does cross-attention happen?	Inside the decoder: decoder queries attend to encoder keys and values.
Is an encoder-decoder autoregressive?	Usually the decoder is autoregressive for generative tasks, although the encoder itself is not.

Designed as a companion chapter to the Transformer and MiniGPT notes. The focus is the thought process: what each component is solving, what flows between blocks, and why one architecture is chosen over another.

CONTENTS

Study Guide Map

Read the first half for concepts and data flow; use the second half for design decisions, worked examples, and revision.

Study Guide Map 2

Three Transformer Architecture Families 4

1.1 A useful mental model . . . 4

What “Autoregressive” Actually Means 5

2.1 Why use this factorization? . . . 5

2.2 Tiny probability example . . . 5

How GPT-Style Autoregression Works 6

3.1 Causal masking . . . 6

3.2 Training arrangement . . . 6

The Complete Data Flow 7

4.1 Source side . . . 7

4.2 Target side . . . 7

What the Encoder Is Optimizing For 8

5.1 Why not immediately compress the whole source into one vector? . . . 8

5.2 What can deeper encoder layers learn? . . . 8

Why the Decoder Has Two Attention Operations 9

6.1 Masked self-attention asks: . . . 9

6.2 Cross-attention asks: . . . 9

6.3 Feed-forward network asks: . . . 9

Queries Come From the Decoder; Keys and Values Come From the Encoder 10

Where Hidden States Finally Become Token Probabilities 11

8.1 Weight tying . . . 11

Teacher Forcing and Shifted Targets 12

9.1 Loss . . . 12

9.2 Why teacher forcing helps . . . 12

Autoregressive Decoding Step by Step 13

10.1 Translation inference example . . . 13

10.2 What can be cached? . . . 13

10.3 Beam search vs sampling . . . 13

Translate “the cat sleeps” **14**

Step A - Encoder . . . 14

Step B - First decoder token . . . 14

Step C - Second decoder token . . . 14

Step D - Continue . . . 14

Two Ways to Solve Text-to-Text Problems **15**

12.1 Same summarization task . . . 15

How to Choose the Architecture **16**

13.1 Ask these questions in order . . . 16

13.2 Then check systems constraints . . . 16

How Much Encoder and Decoder Should We Stack? **17**

14.1 What depth buys you . . . 17

14.2 What width buys you . . . 17

14.3 What heads buy you . . . 17

Apply the Thought Process to Real Tasks **18**

15.1 Encoder-decoder is a general pattern, not only text-to-text . . . 18

15.2 Retrieval-augmented generation . . . 18

Encoder-Decoder Transformer in One Screen **19**

16.1 Cross-attention inside one decoder block . . . 19

Questions Students Usually Ask **20**

17.1 The fastest diagnostic question . . . 20

Everything on One Page **21**

Three subjective practice questions . . . 21

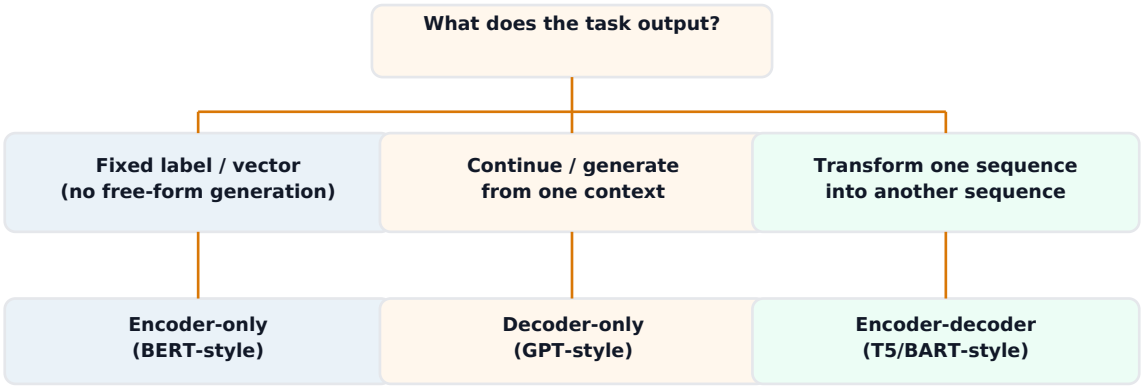
Recommended lecture sequence. Start with autoregressive probability factorization, compare the three architecture families, then trace one translation example through encoder self-attention, decoder masked self-attention, cross-attention, the vocabulary projection, teacher forcing, and inference.

1 - BIG PICTURE

Three Transformer Architecture Families

The same attention building blocks can be arranged for understanding, generation, or input-to-output transformation.

Architecture	Can see both directions?	Generates token-by-token?	Typical use
Encoder-only	Yes, within the input	Not naturally	Classification, tagging, embeddings, retrieval
Decoder-only	No; causal mask	Yes	Chat, completion, code generation, open-ended generation
Encoder-decoder	Encoder: yes; decoder: causal	Yes, in decoder	Translation, summarization, structured text-to-text



Do not mix up two axes. "Encoder vs decoder" is about architectural roles. "Autoregressive vs non-autoregressive" is about the probability/generation process. A decoder-only GPT is autoregressive; the decoder of a translation Transformer is also autoregressive.

1.1 A useful mental model

- Encoder: read the complete source and build a rich representation of what it contains.
- Decoder: decide what target token should come next from the target prefix it has already seen.
- Cross-attention: let the decoder look back into the encoded source whenever it needs source information.
- Output head: convert a decoder hidden state into one logit for every vocabulary token.

2 - AUTOREGRESSIVE MODELING

What “Autoregressive” Actually Means

The model turns sequence probability into a chain of conditional next-token predictions.

For a target sequence y1, y2, ..., yT, the chain rule gives:

```
p(y1, y2, ..., yT | x) = product_t p(yt | y1...y(t-1), x)
```

If there is no separate source x, as in a decoder-only language model:

```
p(x1, x2, ..., xT) = product_t p(xt | x1...x(t-1))
```

> Interpretation. The model never has to predict the entire paragraph in one shot. It solves a repeated simpler problem: given the available context, assign probabilities to the next token.

2.1 Why use this factorization?

- It converts variable-length sequence generation into standard multiclass classification at every position.
- The same parameters are reused at every position and every generation step.
- Training can evaluate all next-token positions in parallel when the true sequence is known.
- At inference time, generation naturally stops when an end token is produced or another stopping rule fires.

2.2 Tiny probability example

Suppose the target is "I love AI" and the model assigns:

Prediction	Probability of correct token
p(I | BOS)	0.60
p(love | BOS, I)	0.50
p(AI | BOS, I, love)	0.80

Then the model probability for this exact sequence is approximately:

```
0.60 x 0.50 x 0.80 = 0.24
```

Training usually minimizes negative log-likelihood, which turns the product into a sum:

```
L = - sum_t log p(correct token at position t)
```

3 - DECODER-ONLY

How GPT-Style Autoregression Works

A single causal stack uses one growing sequence as both the context representation and the generation history.



3.1 Causal masking

The decoder must not see the token it is trying to predict. A causal mask makes the attention matrix lower triangular: position t can attend only to positions <= t.

Query position	Allowed keys
1	1
2	1, 2
3	1, 2, 3
4	1, 2, 3, 4

3.2 Training arrangement

If the token sequence is:

```
[BOS, I, love, AI, EOS]
```

we can use shifted input/target tensors:

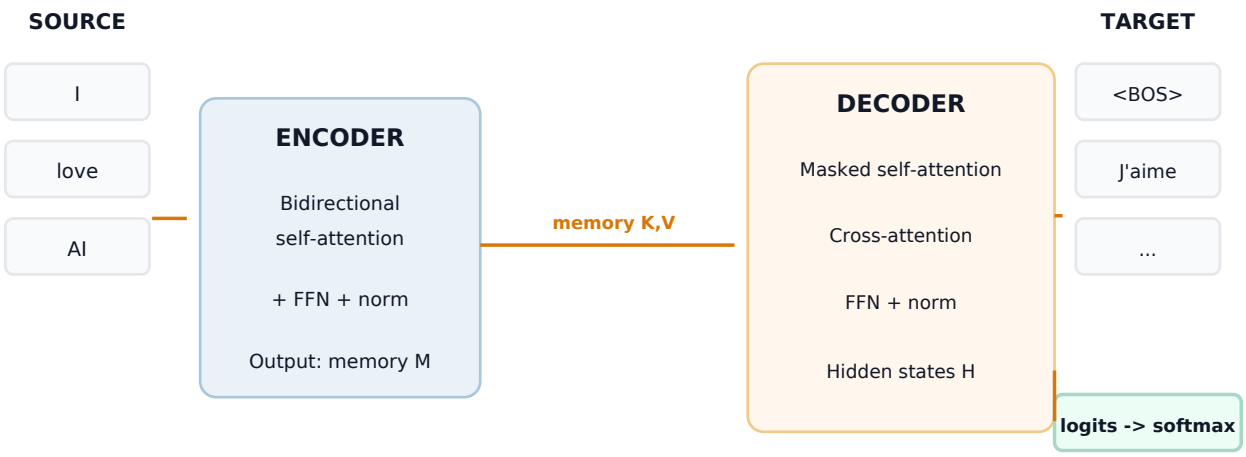
Model input	Target to predict
BOS	I
BOS, I	love
BOS, I, love	AI
BOS, I, love, AI	EOS

Parallel training trick. Although inference is sequential, training feeds the full known sequence at once. The causal mask ensures each position behaves as though future tokens did not exist.

4 - ENCODER-DECODER

The Complete Data Flow

The source and the target have separate roles and separate attention patterns.



4.1 Source side

The source sequence x has length S. After token/position encoding, the encoder applies bidirectional self-attention and feed-forward transformations. Its final output is a matrix:

```
M = Encoder(x), M shape = [B, S, d_model]
```

M is often called encoder memory. Every row is a contextual representation of one source position after information has been mixed across the source.

4.2 Target side

The decoder receives the already-known target prefix. Its masked self-attention builds target-side context. Cross-attention then lets each decoder position retrieve useful information from M.

```
H = Decoder(y_prefix, M), H shape = [B, T, d_model]
```

The key correction. The encoder output is usually not attention scores or vocabulary logits. It is a sequence of contextual vectors. Those vectors become the keys and values used by decoder cross-attention.

5 - ENCODER DESIGN

What the Encoder Is Optimizing For

The encoder has one job: make the source easy for the decoder or downstream head to use.

A typical pre-normalization encoder block can be summarized as:

```
X1 = X + SelfAttention(LayerNorm(X))
X2 = X1 + FFN(LayerNorm(X1))
```

Because the entire source is already known, encoder self-attention is normally bidirectional: every source token may attend to every non-padding source token.

5.1 Why not immediately compress the whole source into one vector?

- Different target words may need different source details at different moments.
- Keeping one vector per source position preserves alignment information.
- Cross-attention can dynamically retrieve a different weighted mixture for each decoder step.
- A long sentence does not have to fit into one fixed-size bottleneck as in early sequence-to-sequence RNN systems.

5.2 What can deeper encoder layers learn?

Earlier layers (rough intuition)	Middle layers	Later layers
Local patterns, morphology, nearby relations	Syntactic and phrase-level dependencies	Task-useful semantic/contextual features

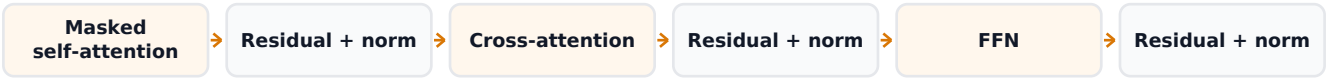
This hierarchy is a useful teaching intuition, not a strict rule: features are distributed and models do not reserve one type of knowledge for one exact layer.

6 - DECODER DESIGN

Why the Decoder Has Two Attention Operations

The decoder must reason over what it has generated and over what the source says.

Each decoder block typically contains:



6.1 Masked self-attention asks:

> Question A. "Given the target tokens I have already generated, what target-side context matters for the next representation?"

6.2 Cross-attention asks:

> Question B. "Given what I am currently trying to generate, which source positions are useful right now?"

6.3 Feed-forward network asks:

> Question C. "After communication through attention, how should I nonlinearly transform the features at this position?"

The decoder therefore alternates between communication (attention) and per-position computation (FFN), while residual connections preserve an information highway through the stack.

7 - CROSS-ATTENTION

Queries Come From the Decoder; Keys and Values Come From the Encoder

Cross-attention is the bridge between the target-generation process and the encoded source.

Let decoder hidden states be H with target length T and encoder memory be M with source length S.

```
Q = H W_Q shape [B, T, d_k]
K = M W_K shape [B, S, d_k]
V = M W_V shape [B, S, d_v]
Attention(H, M) = softmax(Q K^T / sqrt(d_k)) V
```

The score matrix has shape [B, heads, T, S]. Each target position obtains a probability distribution over source positions.

Cross-attention scores

	I	love	AI
Je	0.15	0.65	0.20
aime	0.10	0.25	0.65
intelligence	0.55	0.30	0.15
artificielle	0.20	0.20	0.60

The numbers above are illustrative. A row shows where one target-side query is looking in the source. A high weight does not mean the model has discovered a human-readable alignment rule; it means that, for this head at this layer, that source representation contributes strongly to the weighted value mixture.

Self-attention vs cross-attention. Self-attention gets Q, K, and V from the same sequence. Cross-attention gets Q from one sequence (decoder) and K/V from another (encoder memory).

8 - THE OUTPUT LAYER

Where Hidden States Finally Become Token Probabilities

The decoder stack produces representations; the language-model head turns them into a vocabulary decision.

Suppose the final decoder hidden state at one target position is h_t in R^(d_model). A linear layer maps it to V vocabulary logits:

```
z_t = h_t W_out + b, W_out shape = [d_model, V]
```

Then:

```
p(next token) = softmax(z_t)
```

Vocabulary token	Illustrative logit	Softmax probability
I	0.2	0.04
love	1.1	0.10
AI	2.9	0.61
EOS	1.9	0.25

The largest probability may be selected greedily, or generation may use temperature, top-k, top-p, beam search, or another decoding strategy.

8.1 Weight tying

Many language models reuse the token embedding matrix in the output projection (or tie closely related weights). This reduces parameters and can improve statistical efficiency. Conceptually, however, the two operations remain different: input lookup maps token IDs to vectors; output projection maps hidden vectors to vocabulary scores.

> Encoder outputs need a head only when the task needs one. For classification, an encoder-only model attaches a classification head. For encoder-decoder generation, the vocabulary head is normally attached to the decoder output, not the encoder memory.

9 - TRAINING

Teacher Forcing and Shifted Targets

Training is parallel because the correct previous target tokens are already known.

Consider source: "I love AI" and target: "J'aime l'IA". During training:

Decoder input position	Correct next-token target
	J'aime
, J'aime	l'
...	IA
full target prefix	

This is called teacher forcing: instead of feeding the decoder its own previous sampled output during training, we provide the true previous target token.

9.1 Loss

If target token at position t is y_t and the model probability is p_t(y_t):

```
L = -(1/T) sum_t log p_t(y_t)
```

Padding positions are normally excluded from this average with a loss mask.

9.2 Why teacher forcing helps

- All target positions can be trained in parallel rather than waiting for sampling step by step.
- Early in training the model is poor; using its own mistakes as the next input would create extremely noisy trajectories.
- The model sees the correct conditional context for each supervised next-token prediction.

Training-inference mismatch. At inference, the true next token is unavailable, so the model must condition on its own previous outputs. This difference is sometimes called exposure bias.

10 - INFERENCE

Autoregressive Decoding Step by Step

The encoder is usually run once; the decoder grows the target one token at a time.



10.1 Translation inference example

Step	Decoder prefix	Model chooses
1	[BOS]	J'aime
2	[BOS, J'aime]	l'
3	[BOS, J'aime, l']	IA
4	[BOS, J'aime, l', IA]	EOS

10.2 What can be cached?

- Encoder memory: source representations M are fixed, so they are computed once.
- Decoder self-attention K/V: previous target keys and values can be cached instead of recomputed.
- Cross-attention K/V: projections of encoder memory can also be reused across decoding steps in many implementations.

Without caching, each new token would repeatedly recompute work for the entire already-generated prefix. KV caching trades memory for lower generation latency.

10.3 Beam search vs sampling

Method	Idea	Good fit
Greedy	Always take highest-probability token	Fast deterministic baseline
Beam search	Keep several high-scoring partial sequences	Translation and constrained sequence tasks
Sampling	Randomly sample from controlled probability mass	Creative/open-ended generation

11 - END-TO-END EXAMPLE

Translate “the cat sleeps”

Trace the exact roles of encoder self-attention, decoder self-attention, cross-attention, and the output head.

Assume source tokens: [the, cat, sleeps]. Target should be [le, chat, dort].

Step A - Encoder

- Convert source IDs to embeddings and add/encode position information.
- Run N encoder blocks with unrestricted source self-attention.
- Produce memory vectors M_the, M_cat, M_sleeps. Each vector now contains contextual information about the entire source sentence.

Step B - First decoder token

- Decoder input is BOS.
- Masked self-attention only sees BOS.
- The BOS-derived query cross-attends to all encoder memory vectors.
- The vocabulary head assigns probabilities; suppose "le" has the highest suitable score.

Step C - Second decoder token

- Decoder prefix is [BOS, le].
- Masked self-attention lets the new position use the already-generated "le".
- Cross-attention may now place more weight on the source representation for "cat".
- The output distribution selects "chat".

Step D - Continue

The same process produces "dort" and eventually EOS. Note that cross-attention can change from step to step: the decoder does not receive one fixed source summary; it queries the source differently for each target position.

> The key conceptual separation. Encoder self-attention answers "how should the source tokens contextualize each other?" Decoder self-attention answers "what have I generated so far?" Cross-attention answers "what source information do I need for the target token I am building now?"

12 - DECODER-ONLY VS ENCODER-DECODER

Two Ways to Solve Text-to-Text Problems

Many modern tasks can be expressed with either architecture, but the information flow is different.

Aspect	Decoder-only	Encoder-decoder
Input handling	Input and output are one causal token stream	Source is encoded separately
Source visibility	Later output tokens can attend to all earlier source tokens	Encoder sees entire source bidirectionally
Cross-attention	No separate cross-attention	Explicit decoder-to-encoder cross-attention
Training interface	Prompt + completion	Source tensor + shifted target tensor
Inference	Grow the single sequence	Encode once, then grow target
Typical strength	Simple universal generative interface	Natural inductive bias for sequence transformation

12.1 Same summarization task

Decoder-only formatting:

```
Article: ... long input ... Summary: -> generate continuation
```

Encoder-decoder formatting:

```
Encoder input = article Decoder output = summary
```

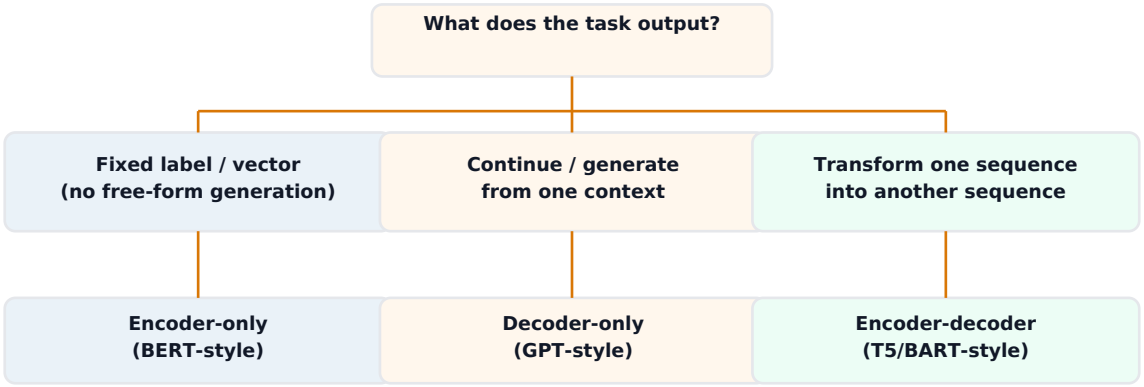
The decoder-only model must represent source and target inside one causal context. The encoder-decoder model can build a bidirectional source representation before target generation begins.

> There is no universal winner. Architecture is one design decision among data scale, objective, model size, latency, tooling, and deployment constraints. Decoder-only systems dominate many general-purpose LLM settings because one next-token interface can absorb many tasks, while encoder-decoder models remain conceptually and practically strong for conditional sequence transformation.

13 - DESIGN THOUGHT PROCESS

How to Choose the Architecture

Start from what information must be available when each prediction is made.



13.1 Ask these questions in order

Question	If YES	Likely direction
Do I only need a fixed label/vector?	No free-form target sequence	Encoder-only
Do I want open-ended continuation/chat?	One growing causal context is natural	Decoder-only
Is there a distinct source and target sequence?	Source should be fully understood before generation	Encoder-decoder
Must each output token align closely to source evidence?	Explicit source memory is useful	Encoder-decoder is attractive
Do I need one universal prompt interface for many tasks?	Task can be phrased as continuation	Decoder-only is attractive

13.2 Then check systems constraints

- Latency: encoder-decoder adds source encoding but can reuse memory; decoder-only may reuse a large prompt KV cache.
- Memory: longer source + target contexts increase attention/cache cost; cross-attention also stores source-side K/V.
- Data: paired source-target data naturally fits encoder-decoder training; massive raw text naturally fits decoder-only next-token pretraining.
- Task flexibility: decoder-only prompting simplifies multi-task interfaces; encoder-decoder can enforce a clearer input/output separation.
- Deployment: existing model ecosystems and serving stacks can outweigh architectural elegance.

14 - DEPTH, WIDTH, AND HEADS

How Much Encoder and Decoder Should We Stack?

There is no formula that says “translation needs exactly N layers”; capacity is chosen under an empirical compute-quality budget.

14.1 What depth buys you

Stacking blocks repeatedly alternates attention-based communication with nonlinear feature transformation. More layers allow more stages of representation refinement and interaction.

14.2 What width buys you

A larger d_model gives each token more feature dimensions. The FFN hidden dimension is often a multiple of d_model, so increasing width can increase parameters and compute quickly.

14.3 What heads buy you

Multiple heads provide multiple learned query/key/value subspaces in the same layer. Heads can specialize in different interaction patterns, but "more heads" is not automatically better if each head becomes too narrow or the model is under-trained.

Design change	Potential benefit	Cost / risk
More encoder layers	Richer source representation	Encoding latency and parameters
More decoder layers	Richer generation reasoning/transformations	Higher per-token latency
Larger d_model	More representational capacity	Quadratic-ish parameter growth across projections/FFNs
More heads	More parallel attention subspaces	Smaller dimension per head if d_model fixed
Longer context	More source/history available	Attention/cache memory and latency

Practical process. Choose a baseline that fits your hardware, train/evaluate it, inspect underfitting vs overfitting and latency, then scale one dimension at a time. Model architecture is an empirical engineering tradeoff, not a hand-derived constant.

15 - ARCHITECTURE CASE STUDIES

Apply the Thought Process to Real Tasks

The goal is to justify the information flow, not memorize product names.

Task	Architecture choice	Reasoning
Sentiment classification	Encoder-only	Need one label; full bidirectional input context is useful.
Code autocomplete	Decoder-only	Output is a continuation of an existing prefix.
General chat assistant	Decoder-only	Conversation + instructions + generated reply fit one autoregressive stream.
Machine translation	Encoder-decoder	Distinct source/target languages; decoder can query source memory while generating.
Abstractive summarization	Encoder-decoder or decoder-only	Both work; separate source representation can be a natural inductive bias.
Image captioning	Encoder-decoder pattern	An image encoder can provide memory; a text decoder generates the caption using cross-attention.
Speech recognition	Encoder-decoder pattern	Audio encoder represents frames; decoder generates text tokens conditioned on encoded audio.

15.1 Encoder-decoder is a general pattern, not only text-to-text

The "encoder" can process another modality. Cross-attention only requires a sequence/set of source representations that the decoder can use as keys and values. That makes the pattern useful for vision-language, speech-to-text, and other conditional generation systems.

15.2 Retrieval-augmented generation

RAG with a decoder-only LLM usually concatenates retrieved text into the prompt rather than adding a classic encoder-decoder cross-attention module. Other architectures can encode retrieved evidence separately. The important design question is still: where is source information represented, and how can each generated token access it?

16 - MINIMAL PSEUDOCODE

Encoder-Decoder Transformer in One Screen

This is conceptual code: it shows tensor flow rather than production implementation details.

```
# src: [B, S] source token IDs
# tgt_in: [B, T] shifted target token IDs

src_x = src_embedding(src) + src_position(src)
tgt_x = tgt_embedding(tgt_in) + tgt_position(tgt_in)

# Encoder: no causal mask on source
memory = encoder(src_x, padding_mask=src_pad_mask)
# memory: [B, S, d_model]

# Decoder:
# 1) causal self-attention over target prefix
# 2) cross-attention: Q from decoder, K/V from memory
h = decoder(
    tgt_x,
    memory,
    causal_mask=tgt_causal_mask,
    source_padding_mask=src_pad_mask
)
# h: [B, T, d_model]

logits = lm_head(h)
# logits: [B, T, vocab_size]

loss = cross_entropy(
    logits.reshape(-1, vocab_size),
    tgt_labels.reshape(-1),
    ignore_index=PAD_ID
)
```

16.1 Cross-attention inside one decoder block

```
# h is the decoder stream, memory is encoder output
q = Wq(h)
k = Wk(memory)
v = Wv(memory)

scores = q @ k.transpose(-2, -1) / sqrt(d_k)
weights = softmax(scores, dim=-1)
context = weights @ v

h = h + Wo(context)
```

Shape check. If source length S=100 and target length T=20, one cross-attention head creates a T x S score matrix: 20 x 100. Decoder self-attention creates a T x T causal score matrix: 20 x 20.

17 - COMMON MISCONCEPTIONS

Questions Students Usually Ask

Use these as oral revision prompts.

Misconception	Correction
"Autoregressive means decoder-only."	No. The decoder of an encoder-decoder model can also be autoregressive.
"The encoder predicts the output sentence first."	No. It creates contextual source vectors; the decoder generates the target.
"Cross-attention feeds attention probabilities into the decoder."	The weighted value output becomes a context vector; probabilities are intermediate weights.
"The decoder can see the whole correct target."	During training the tensor contains it, but the causal mask blocks future positions from each query.
"Teacher forcing means the model copies the answer."	It receives correct previous tokens but must still predict the next token at every position.
"More encoder/decoder layers always improve the model."	Only when data, optimization, compute, and task complexity support the extra capacity.
"Encoder-decoder has two language models."	Not necessarily. The encoder is a representation network; the autoregressive language-model behavior lives in the decoder output distribution.

17.1 The fastest diagnostic question

Ask: "At this exact prediction position, what information is the model legally allowed to see, and through which attention path can it reach that information?" This question resolves most mask, self-attention, and cross-attention confusion.

18 - REVISION SHEET

Everything on One Page

A compact mental model for exams, interviews, and teaching.

Concept	Remember this
Autoregressive	Factor sequence probability into next-token conditional probabilities.
Encoder	Bidirectionally contextualize source tokens -> memory M.
Decoder self-attention	Causal communication among target-prefix positions.
Cross-attention	Decoder Q attends to encoder K/V.
FFN	Per-position nonlinear feature transformation after attention communication.
Residual	Preserve a direct information/gradient path through deep stacks.
LayerNorm / RMSNorm	Keep representation scales stable enough for deep optimization.
Output head	d_model hidden vector -> V vocabulary logits.
Teacher forcing	Train each next-token prediction with the true previous target prefix.
Inference	Encode once, then generate autoregressively until EOS/stop.

Three subjective practice questions

1. Autoregressive reasoning: Explain why an autoregressive model factorizes p(y1...yT) into conditional probabilities. For a 3-token output with correct-token probabilities 0.8, 0.5, and 0.25, calculate the sequence probability and explain why training usually uses log probabilities rather than multiplying probabilities directly.

2. Encoder-decoder data flow: For a translation model with source length S=12, target length T=7, d_model=256, and 8 heads, state the shapes of encoder memory, decoder hidden states, per-head dimension, decoder self-attention score matrix, and cross-attention score matrix. Explain what Q, K, and V come from in cross-attention.

3. Architecture design: You must build (a) a sentiment classifier, (b) a code autocomplete system, and (c) an English-to-Hindi translator. Choose encoder-only, decoder-only, or encoder-decoder for each. Justify what information must be visible at prediction time, what masks are required, and where the final logits come from.

One-sentence summary. An encoder-decoder Transformer first builds a bidirectional source memory, then an autoregressive decoder repeatedly combines its target-prefix state with source information through cross-attention and projects the resulting hidden state into next-token probabilities.